

Лабораторная работа № 5

Аппарат сетей Петри: задача обедающих философов

Куокконен Дарина Андреевна

2026-08-31

Аннотация

Построены две сети Петри для задачи обедающих философов: классическая сеть с последовательным захватом вилок и сеть с арбитром. Реализованы стохастическое моделирование по прямому алгоритму Гиллеспи и непрерывная mass-action аппроксимация. На базовой траектории классическая система заблокировалась после пяти событий, тогда как сеть с арбитром выполнила 31 цикл приёма пищи. Серия из 960 прогонов подтвердила отсутствие глобальной блокировки у модифицированной сети.

Российский университет дружбы народов имени Патриса Лумумбы.
Учебная группа: НФИбд-01-23.

Содержание

1. Цель и постановка задачи
2. Теоретические основы сетей Петри
3. Формализация задачи обедающих философов
4. Организация вычислительного проекта
5. Реализация модели
6. Базовый стохастический эксперимент
7. Анализ сохранённых CSV
8. Анимация маркировки
9. Детерминированная аппроксимация
10. Параметрический эксперимент
11. Литературное программирование и Jupyter
12. Проверка реализации
13. Выводы
14. Источники

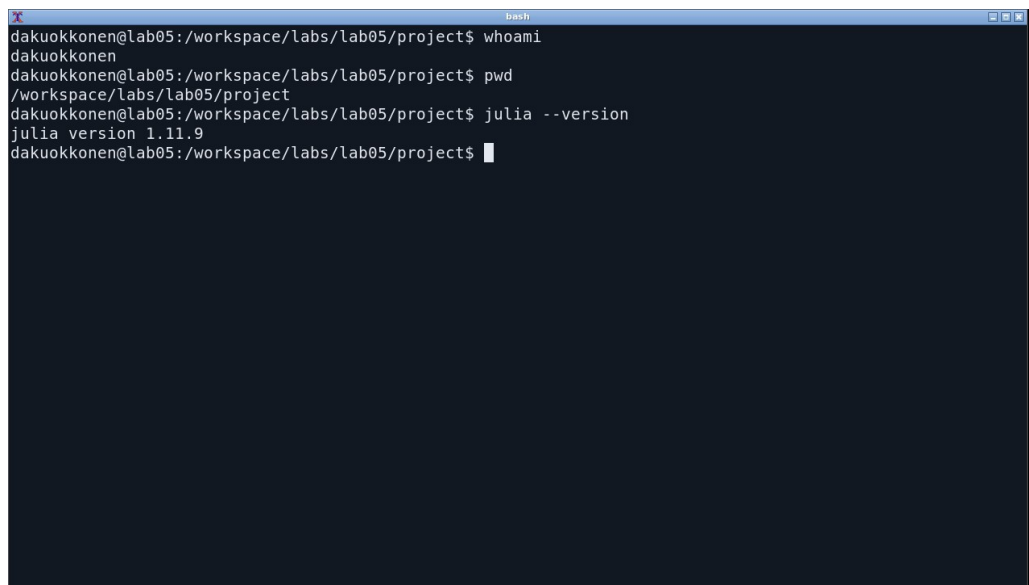
1. Цель и постановка задачи

Цель работы — исследовать взаимную блокировку при конкурентном доступе пяти процессов к пяти разделяемым ресурсам и оценить эффективность арбитра средствами сетей Петри.

Для достижения цели выполнены следующие задачи:

1. формально описаны позиции, переходы, дуги и начальная маркировка;
2. построены классическая сеть и сеть с дополнительной позицией-арбитром;
3. реализован прямой стохастический алгоритм Гиллеспи;
4. добавлена непрерывная mass-action аппроксимация системы;
5. определены критерий deadlock, число завершённых приёмов пищи и индекс равномерности Джейна;
6. проведены базовый и параметрический эксперименты, построена анимация;
7. отдельно проанализированы сохранённые CSV без повторного моделирования;
8. для каждого сценария получены Julia-скрипт, выполненный блокнот и QMD;
9. реализация проверена автоматическими тестами.

Исходное состояние рабочего окружения показано на [рис. 1](#).

A screenshot of a terminal window with a dark background and light text. The window title is 'bash'. The user 'dakuokkonen' is at a machine named 'lab05' in the directory '/workspace/labs/lab05/project'. They have executed three commands: 'whoami' (output: dakuokkonen), 'pwd' (output: /workspace/labs/lab05/project), and 'julia --version' (output: julia version 1.11.9). The prompt is currently at the end of the third command line.

```
dakuokkonen@lab05:/workspace/labs/lab05/project$ whoami
dakuokkonen
dakuokkonen@lab05:/workspace/labs/lab05/project$ pwd
/workspace/labs/lab05/project
dakuokkonen@lab05:/workspace/labs/lab05/project$ julia --version
julia version 1.11.9
dakuokkonen@lab05:/workspace/labs/lab05/project$
```

Рисунок 1: Проверка пользователя, версии Julia и каталога проекта

2. Теоретические основы сетей Петри

Сеть Петри задаётся четвёркой

$$N=(P,T,F,M_0),$$

где P — конечное множество позиций, T — множество переходов, $F \subseteq (P \times T) \cup (T \times P)$ — отношение дуг, а M_0 — начальная маркировка. Формализм был предложен К. А. Петри для описания взаимодействующих автоматов и позднее стал одним из основных средств анализа параллельных систем [1,2].

Для матриц входных и выходных дуг A и B вводится матрица инцидентности

$$C=B-A.$$

Переход t_j разрешён в маркировке M , если для каждой позиции выполняется $M_i \geq A_{ij}$. После срабатывания маркировка изменяется атомарно:

$$M' = M + C_{\cdot j}.$$

Глобальный deadlock фиксируется, когда множество разрешённых переходов пусто. Для рассматриваемой сети это означает, что ни один философ не может получить вторую вилку и ни один уже не ест.

3. Формализация задачи обедающих философов

Для каждого философа i определены три позиции состояния: Think_ i , Hungry_ i и Eat_ i . Позиция Hungry_ i означает, что философ уже захватил левую вилку. Каждая вилка представлена ресурсной позицией Fork_ i .

Таблица 1: Размерность и начальная маркировка сетей

Элемент	Классическая сеть	Сеть с арбитром
Философы	5	5
Позиции	20	21
Переходы	15	15
Фишки в позициях Think	5	5
Фишки в позициях Fork	5	5
Фишки в позиции Arbiter	—	4

Переходы GetLeft_ i , GetRight_ i и PutForks_ i соответственно моделируют захват левой вилки, захват правой вилки и освобождение обеих вилок. В классической сети все пять переходов

GetLeft_i могут сработать до любого GetRight_i: возникает циклическое ожидание. В модифицированной сети GetLeft_i дополнительно потребляет разрешение из позиции Arbiter, в которой находятся только $N - 1$ фишек. Поэтому хотя бы один философ остаётся вне цикла захвата и одна из цепочек может завершиться. Структура изменений маркировки представлена матрицей на [рис. 2](#).

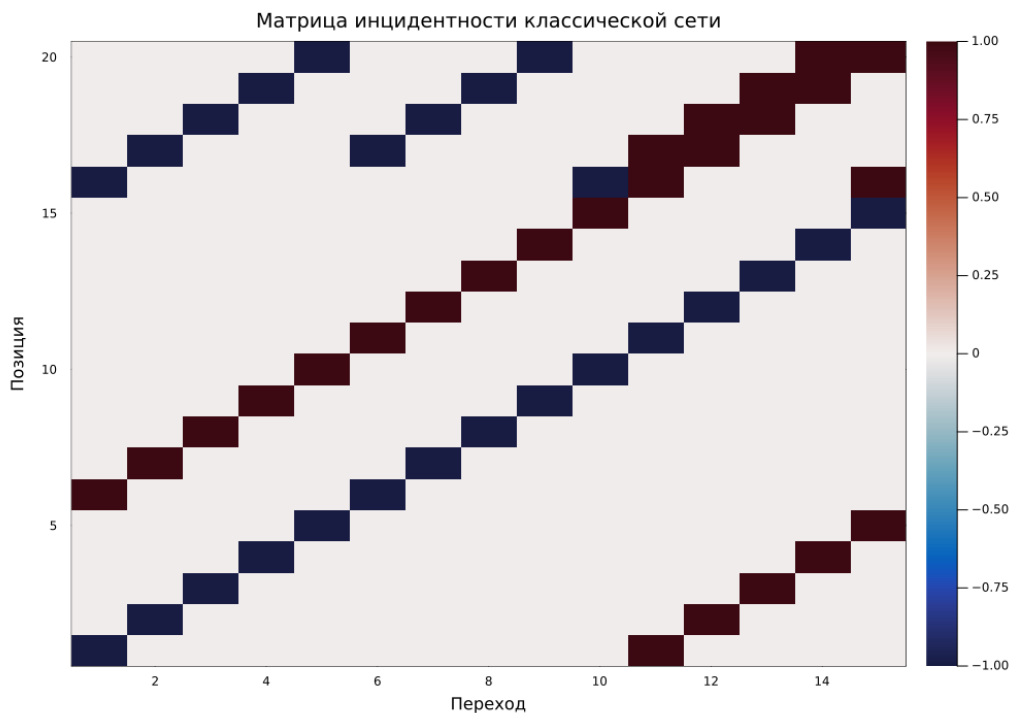


Рисунок 2: Матрица инцидентности классической сети

4. Организация вычислительного проекта

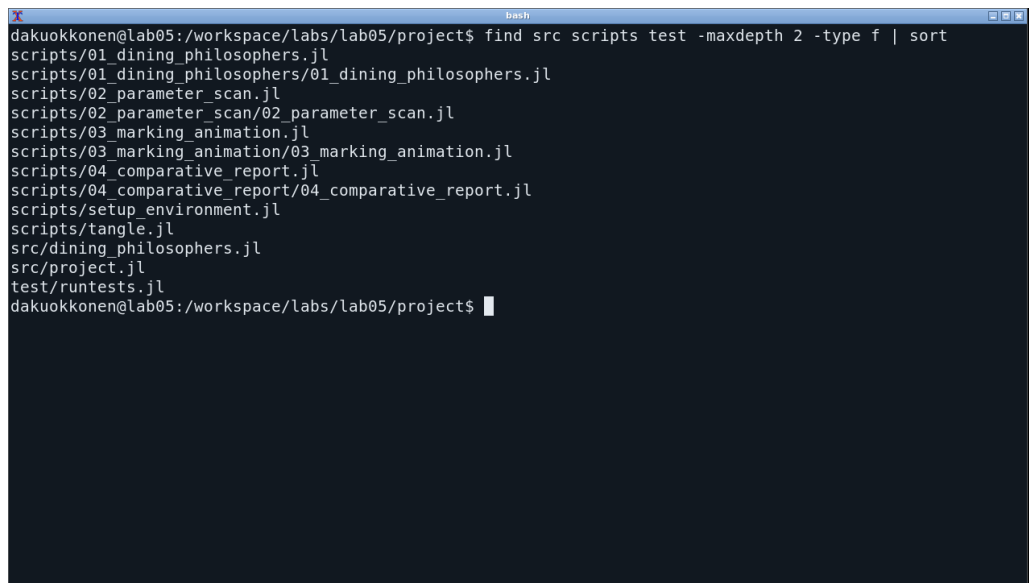
Расчёты выполнены в Julia 1.11.9 внутри изолированного окружения. Версии пакетов закреплены в Project.toml и Manifest.toml.

Таблица 2: Состав Julia-проекта

Компонент	Назначение
src/dining_philosophers.jl	структуры сети, правила срабатывания и оба алгоритма
scripts/01_dining_philosophers.jl	базовый стохастический и ODE-эксперимент
scripts/02_parameter_scan.jl	960 прогонов по числу философов и интенсивности
scripts/03_marking_animation.jl	анимация классической сети при $N=3$ и сравнение с арбитром

Компонент	Назначение
scripts/04_comparative_report.jl	чтение сохранённых CSV и сравнение позиций Eat
scripts/tangle.jl	получение .jl, QMD и IPYNB из литературного кода
test/runtests.jl	структурные, динамические и численные проверки

Фактический состав каталогов показан на [рис. 3](#).



```

dakuokkonen@lab05:/workspace/labs/lab05/project$ find src scripts test -maxdepth 2 -type f | sort
scripts/01_dining_philosophers.jl
scripts/01_dining_philosophers/01_dining_philosophers.jl
scripts/02_parameter_scan.jl
scripts/02_parameter_scan/02_parameter_scan.jl
scripts/03_marking_animation.jl
scripts/03_marking_animation/03_marking_animation.jl
scripts/04_comparative_report.jl
scripts/04_comparative_report/04_comparative_report.jl
scripts/setup_environment.jl
scripts/tangle.jl
src/dining_philosophers.jl
src/project.jl
test/runtests.jl
dakuokkonen@lab05:/workspace/labs/lab05/project$

```

Рисунок 3: Исходники, данные и производные форматы проекта

5. Реализация модели

5.1 Построение входных и выходных дуг

В реализации входные и выходные дуги хранятся отдельно. Такой способ не теряет информацию при наличии петель и позволяет одинаково использовать сеть в дискретном и непрерывном алгоритмах.

```

input[think, get_left] = 1
input[left_fork, get_left] = 1
output[hungry, get_left] = 1
if arbiter
    input[arbiter_place, get_left] = 1
end

input[hungry, get_right] = 1
input[right_fork, get_right] = 1
output[eat, get_right] = 1

```

```

input[eat, put_forks] = 1
output[think, put_forks] = 1
output[left_fork, put_forks] = 1
output[right_fork, put_forks] = 1
if arbiter
    output[arbiter_place, put_forks] = 1
end

```

Листинг 1 — Построение дуг одного философа в `src/dining_philosophers.jl`

5.2 Проверка и срабатывание перехода

Разрешённость перехода вычисляется непосредственно по матрице входных дуг. Изменение маркировки выполняется одной операцией с колонкой матрицы инцидентности.

```

function enabled_transitions(net::PetriNet,
    marking::AbstractVector{<:Real}; tol=1e-10)
    length(marking) == nplaces(net) ||
        throw(ArgumentError("marking length mismatch"))
    [j for j in 1:ntransitions(net)
        if all(marking[i] + tol >= net.input[i, j]
            for i in 1:nplaces(net))]
end

function fire_transition!(net::PetriNet, marking::Vector{Int},
    transition::Integer)
    transition in enabled_transitions(net, marking) ||
        throw(ArgumentError("transition is not enabled"))
    marking .+= incidence(net)[:, transition]
    marking
end

```

Листинг 2 — Семантика срабатывания в `src/dining_philosophers.jl`

5.3 Прямой алгоритм Гиллеспи

В каждый момент выбираются разрешённые переходы. Время до следующего события имеет экспоненциальное распределение с параметром, равным сумме их интенсивностей. Затем переход выбирается пропорционально собственной интенсивности. Это соответствует прямому алгоритму Гиллеспи [3].

```

enabled = enabled_transitions(net, marking)
isempty(enabled) && (deadlock = true; break)
total_rate = sum(rates[enabled])
dt = randexp(rng) / total_rate

```

```

threshold = rand(rng) * total_rate

cumulative = 0.0
for transition in enabled
    cumulative += rates[transition]
    if threshold <= cumulative
        chosen = transition
        break
    end
end
fire_transition!(net, marking, chosen)

```

Листинг 3 — Выбор события в src/dining_philosophers.jl

6. Базовый стохастический эксперимент

Для обеих сетей использованы $N=5$, горизонт $t_{max}=50$ и одинаковое начальное число генератора. Интенсивности переходов равны 2,0 для захвата левой вилки, 1,0 для захвата правой и 1,4 для освобождения ресурсов.

6.1 Документация базового сценария

Ниже подключена документация, автоматически сформированная из scripts/01_dining_philosophers.jl. Листинги охватывают весь сценарий: от активации проекта до сохранения графиков.

6.1.1 Обедающие философы: базовый эксперимент

Цель: сопоставить классическую сеть Петри и сеть с арбитром для пяти философов, проверить появление взаимной блокировки и сравнить стохастическую траекторию с непрерывной аппроксимацией.

```

using DrWatson
@quickactivate "project"
ENV["GKSwstype"] = "100"
using CSV, DataFrames, Plots, Random, Statistics
include(sourcedir("dining_philosophers.jl"))
using .DiningPhilosophers

```

6.1.1.1 Параметры эксперимента

Пять философов используют одинаковые интенсивности переходов.

```

const N = 5
const TMAX = 50.0
const RATES = default_rates(N; left_rate=2.0, right_rate=1.0,
release_rate=1.4)
const IMAGE_DIR = projectdir("../", "image")

```

```
const DATA_DIR = datadir("01_dining_philosophers")
mkpath(IMAGE_DIR)
mkpath(DATA_DIR)
```

6.1.1.2 Построение и стохастическое выполнение сетей

Один seed позволяет повторить каждую базовую траекторию.

```
classical_net, classical_initial = build_classical_network(N)
arbiter_net, arbiter_initial = build_arbiter_network(N)

classical = simulate_stochastic(classical_net, classical_initial,
TMAX;
    rates=RATES, rng=MersenneTwister(202605))
arbiter = simulate_stochastic(arbiter_net, arbiter_initial, TMAX;
    rates=RATES, rng=MersenneTwister(202605))

CSV.write(joinpath(DATA_DIR, "classical_trajectory.csv"),
classical.trajectory)
CSV.write(joinpath(DATA_DIR, "arbiter_trajectory.csv"),
arbiter.trajectory)
```

6.1.1.3 Детерминированная аппроксимация

Непрерывные значения не являются целочисленной маркировкой сети.

```
ode_classical = simulate_ode(classical_net, classical_initial, 20.0;
    rates=RATES, saveat=0.1)
ode_arbiter = simulate_ode(arbiter_net, arbiter_initial, 20.0;
    rates=RATES, saveat=0.1)
CSV.write(joinpath(DATA_DIR, "classical_ode.csv"), ode_classical)
CSV.write(joinpath(DATA_DIR, "arbiter_ode.csv"), ode_arbiter)
```

6.1.1.4 Сводные показатели

Сопоставляем блокировку, число событий, приёмы пищи и индекс Джейна.

```
summary = DataFrame(
    variant=["Классическая сеть", "Сеть с арбитром"],
    deadlock=[classical.deadlock, arbiter.deadlock],
    stop_time=[classical.stop_time, arbiter.stop_time],
    events=[classical.events, arbiter.events],
    completed_meals=[sum(classical.meal_counts),
sum(arbiter.meal_counts)],
    fairness=[jain_fairness(classical.meal_counts),
jain_fairness(arbiter.meal_counts)],
)
CSV.write(joinpath(DATA_DIR, "summary.csv"), summary)
```

```
println("=== Обедающие философы: базовый эксперимент ===")
println("Философов: $N, горизонт: $TMAX")
show(summary; allrows=true, allcols=true)
println()
println("Завершённые приёмы пищи, классическая сеть: $
(classical.meal_counts)")
println("Завершённые приёмы пищи, сеть с арбитром: $
(arbiter.meal_counts)")
```

6.1.1.5 Эволюция маркировок

Результаты сохраняются в общий каталог изображений лабораторной.

```
default(fontfamily="DejaVu Sans", linewidth=2.1, framestyle=:box,
        gridalpha=0.22, legendfontsize=8, guidefontsize=11,
        titlefontsize=13,
        left_margin=7Plots.mm, bottom_margin=5Plots.mm)

function marking_panels(trajjectory, N, variant)
    panels = map(("Think", "Hungry", "Eat", "Fork")) do group
        columns = [Symbol("$(group)_$i") for i in 1:N]
        plot(trajjectory.time, Matrix(trajjectory[:, columns]);
            label=permutedims(["$group $i" for i in 1:N]),
            seriestype=:steppost, xlabel="Время", ylabel="Фишки",
            title="$variant: $group", ylims=(-0.08, 1.08),
            legend=:outright)
    end
    plot(panels...; layout=(2, 2), size=(1400, 900))
end

p1 = marking_panels(classical.trajectory, N, "Классическая сеть")
savefig(p1, joinpath(IMAGE_DIR, "01-plot.png"))

p2 = marking_panels(arbiter.trajectory, N, "С арбитром")
savefig(p2, joinpath(IMAGE_DIR, "02-plot.png"))
```

6.1.1.6 Сравнение непрерывных решений

```
classical_ode_aggregate = aggregate_trajectory(ode_classical, N)
arbiter_ode_aggregate = aggregate_trajectory(ode_arbiter, N)
p4 = plot(classical_ode_aggregate.time, classical_ode_aggregate.Eat;
    label="классическая сеть", xlabel="Время", ylabel="Сумма Eat",
    title="Детерминированная mass-action аппроксимация", size=(1000,
620))
plot!(p4, arbiter_ode_aggregate.time, arbiter_ode_aggregate.Eat;
    label="с арбитром")
savefig(p4, joinpath(IMAGE_DIR, "04-plot.png"))
```

6.1.1.7 Матрица инцидентности

```
matrix_plot = heatmap(classical_net.output - classical_net.input;  
    xlabel="Переход", ylabel="Позиция", title="Матрица инцидентности  
классической сети",  
    color=:balance, size=(1000, 700))  
savefig(matrix_plot, joinpath(IMAGE_DIR, "05-plot.png"))
```

Классическая сеть заканчивает работу в маркировке, где каждый философ удерживает одну вилку. Арбитр ограничивает число одновременно начавших захват ресурсов и тем самым разрывает условие циклического ожидания.

6.2 Результаты базового расчёта

Таблица 3: Результаты базового эксперимента

Показатель	Классическая сеть	Сеть с арбитром
Deadlock	да	нет
Время последнего события	0,5973	48,9047
Число событий	5	98
Завершённые приёмы пищи	0	31
Индекс Джейна	0,0000	0,9109

Классическая траектория заблокировалась сразу после пяти срабатываний: каждый философ перешёл из Think_i в Hungry_i и забрал левую вилку. Ни один GetRight_i после этого не был разрешён. На [рис. 4](#) видно накопление всех пяти фишек в позициях Hungry и исчезновение свободных вилок. Для каждого варианта построены четыре панели: Think, Hungry, Eat и Fork. Каждая линия соответствует отдельному философу или вилке, поэтому графики сохраняют информацию о распределении ресурсов, а не только их сумме.

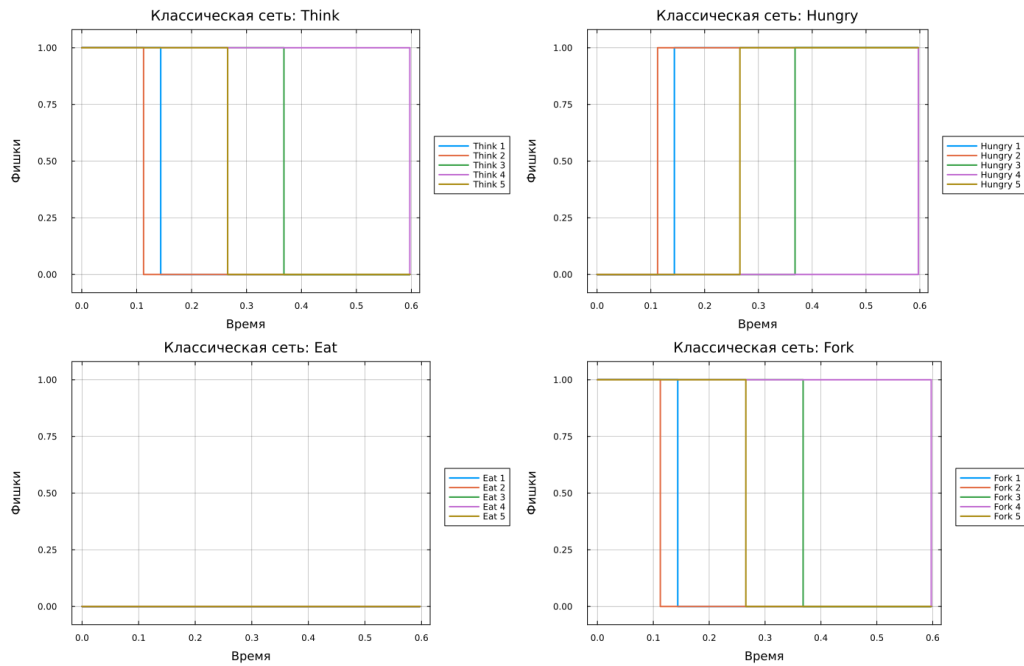


Рисунок 4: Маркировка классической сети до deadlock

Арбитр ограничил число одновременно начатых захватов четырьмя. За тот же горизонт выполнено 98 переходов и завершён 31 цикл питания. Распределение циклов между философами [5, 10, 5, 5, 6] даёт индекс Джейна 0,9109: доступ остаётся достаточно равномерным, хотя случайная траектория не обязана быть полностью симметричной. На [рис. 5](#) состояния продолжают чередоваться на всём горизонте эксперимента.

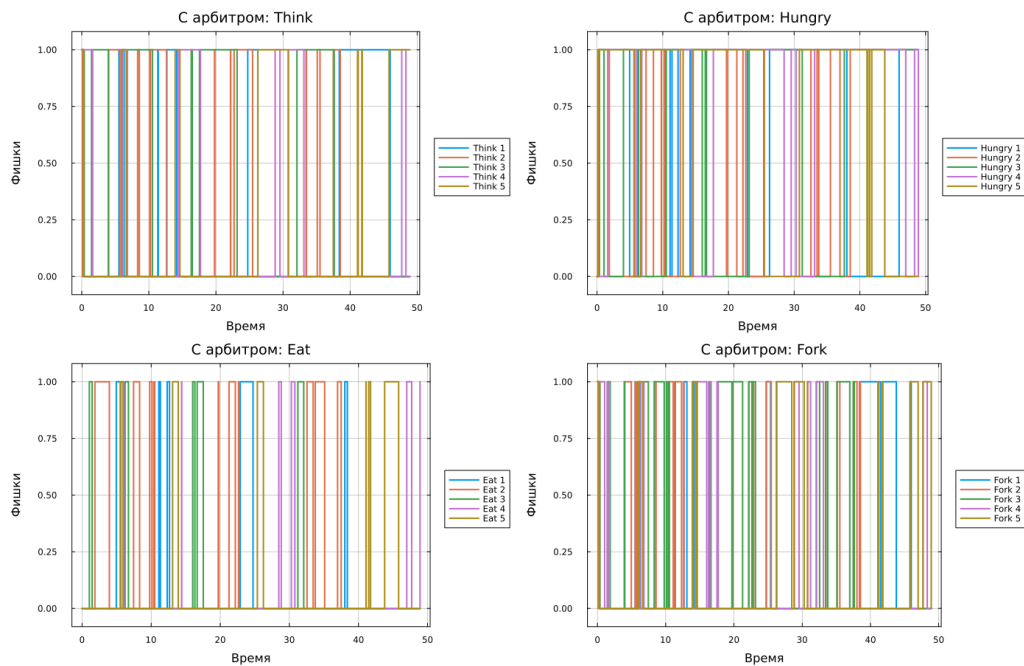


Рисунок 5: Маркировка сети с арбитром

7. Анализ сохранённых CSV

Сценарий `scripts/04_comparative_report.jl` читает результаты базового запуска, не генерируя новые траектории. Это разделяет получение данных и их анализ: один и тот же набор CSV можно повторно исследовать независимо от генератора случайных чисел. Перед построением графика проверяются имена колонок `Eat_1-Eat_5` и порядок времени.

7.1 Документация сравнительного сценария

Полный код и пояснения из `scripts/04_comparative_report.jl` подключены через автоматически полученный QMD-фрагмент.

7.1.1 Сравнительный анализ сохранённых траекторий

Этот сценарий не запускает модель заново. Входные CSV должны быть получены базовым экспериментом `01_dining_philosophers.jl`.

```
using DrWatson
@quickactivate "project"
ENV["GKSwstype"] = "100"
using CSV, DataFrames, Plots, Statistics
include(sourcedir("dining_philosophers.jl"))
using .DiningPhilosophers
```

7.1.1.1 Загрузка ранее полученных данных

```
const N = 5
const INPUT_DIR = datadir("01_dining_philosophers")
const OUTPUT_DIR = datadir("04_comparative_report")
const IMAGE_DIR = projectdir(".", "image")
classic_file = joinpath(INPUT_DIR, "classical_trajectory.csv")
arbiter_file = joinpath(INPUT_DIR, "arbiter_trajectory.csv")
for file in (classic_file, arbiter_file)
    isfile(file) || error("Сначала выполните базовый эксперимент:
отсутствует $(basename(file))")
end
classical = CSV.read(classic_file, DataFrame)
arbiter = CSV.read(arbiter_file, DataFrame)
mkpath(OUTPUT_DIR)
mkpath(IMAGE_DIR)
```

7.1.1.2 Сопоставление позиций *Eat*

```
eat_columns = [Symbol("Eat_$i") for i in 1:N]
for trajectory in (classical, arbiter)
    all(name -> name in propertynames(trajectory), eat_columns) ||
        error("В CSV отсутствуют необходимые столбцы Eat")
    issorted(trajectory.time) || error("Временные отметки должны
```

```

быть упорядочены")
end
default(fontfamily="DejaVu Sans", linewidth=2, framestyle=:box,
        gridalpha=0.22, left_margin=7Plots.mm, bottom_margin=5Plots.mm)
labels = permutedims(["Философ $i" for i in 1:N])
p1 = plot(classical.time, Matrix(classical[:, eat_columns]));
        label=labels, seriestype=:steppost, xlabel="Время", ylabel="Eat
(0/1)",
        ylims=(-0.08, 1.08), title="Классическая сеть",
legend=:outright)
p2 = plot(arbiter.time, Matrix(arbiter[:, eat_columns]));
        label=labels, seriestype=:steppost, xlabel="Время", ylabel="Eat
(0/1)",
        ylims=(-0.08, 1.08), title="Сеть с арбитром",
legend=:outright)
comparison = plot(p1, p2; layout=(2, 1), size=(1100, 850))
savefig(comparison, joinpath(IMAGE_DIR, "03-plot.png"))

```

7.1.1.3 Проверка последней маркировки

```

classic_net, _ = build_classical_network(N)
arbiter_net, _ = build_arbiter_network(N)
classic_last = Int[last(classical)[name] for name in
classic_net.place_names]
arbiter_last = Int[last(arbiter)[name] for name in
arbiter_net.place_names]
summary = DataFrame(variant=["Классическая", "С арбитром"],
        rows=[nrow(classical), nrow(arbiter)],
        last_event=[last(classical.time), last(arbiter.time)],
        deadlock=[detect_deadlock(classic_net, classic_last),
detect_deadlock(arbiter_net, arbiter_last)],
        max_eating=[maximum(sum(Matrix(classical[:, eat_columns]));
dims=2)),
maximum(sum(Matrix(arbiter[:, eat_columns]));
dims=2))])
CSV.write(joinpath(OUTPUT_DIR, "summary.csv"), summary)
show(summary; allrows=true, allcols=true)
println()

```

Нулевое значение Eat само по себе не доказывает deadlock. Проверка множества разрешённых переходов отличает взаимную блокировку от временной паузы питания.

7.2 Результаты анализа CSV

Сопоставление маркеров Eat_i показывает принципиальное различие режимов: классическая сеть не успевает завершить ни одного приёма пищи, тогда как в модифицированной сети философы последовательно получают обе вилки (рис. 6).

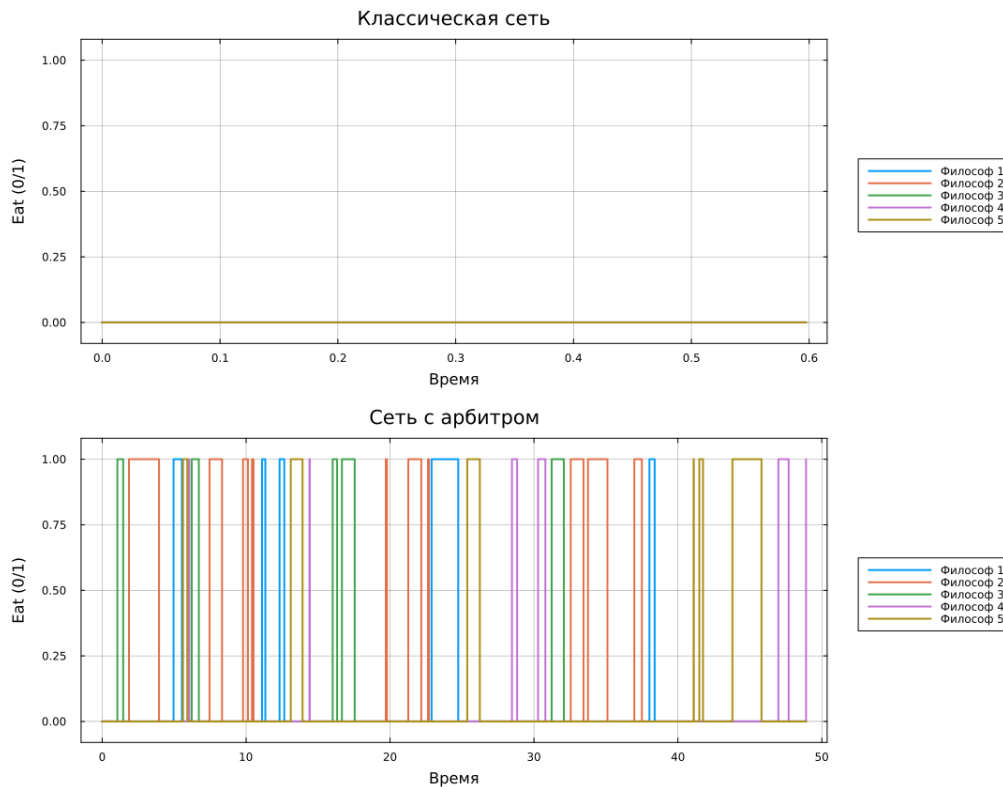


Рисунок 6: Динамика позиций *Eat* для двух сетей

Классический CSV содержит 6 строк: начальную маркировку и 5 событий. CSV сети с арбитром содержит 99 строк: начальное состояние и 98 событий. Последние временные отметки равны 0,597319 и 48,904657 соответственно. Горизонт моделирования составляет 50, но строка для него не добавляется искусственно, если в этот момент не произошло событие.

В обоих файлах значения *Eat_i* целочисленные. Максимальное число одновременно обедающих равно 0 и 1 для данной пары траекторий. Само отсутствие фишек в *Eat* ещё не доказывает deadlock: итоговая маркировка проверяется по всем входным дугам сети. Таблица проверки сохранена в `data/04_comparative_report/summary.csv`.

Выполнение отдельного блокнота анализа показано на [рис. 7](#): ячейки читают уже сохранённые таблицы и строят сравнительный график.

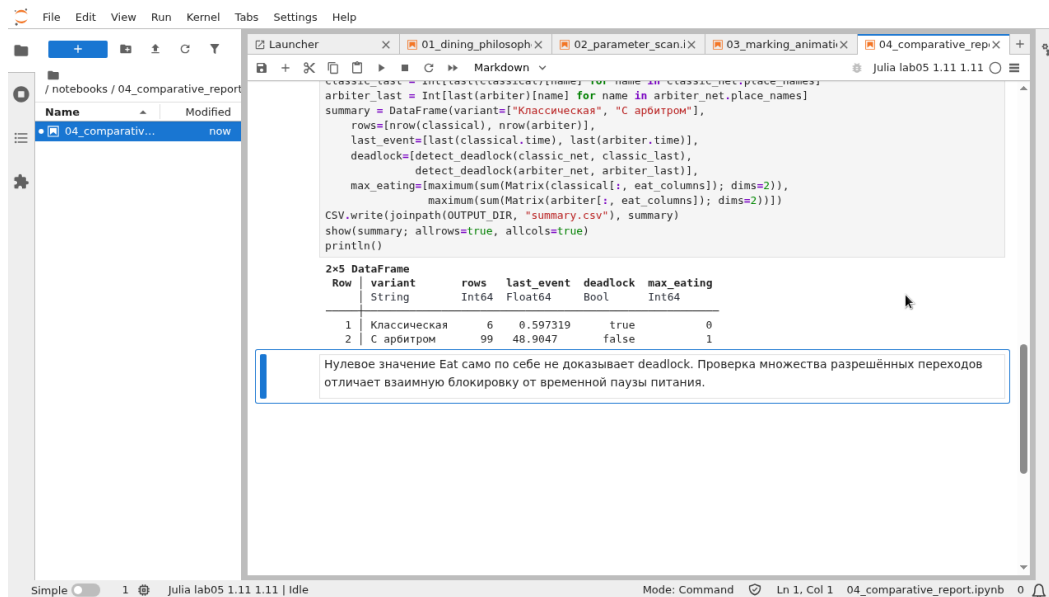


Рисунок 7: Анализ сохранённых CSV в Jupyter

8. Анимация маркировки

Отдельный сценарий `scripts/03_marking_animation.jl` воспроизводит анимационный эксперимент методички: классическая сеть из трёх философов, горизонт 30, единичные интенсивности переходов и начальное состояние генератора 123. Каждый кадр показывает число фишек во всех позициях после очередного события. GIF воспроизводится с частотой 5 кадров в секунду; это скорость просмотра, а не масштаб модельного времени. Фактическое время события указано в заголовке кадра.

8.1 Документация анимационного сценария

Полный текст сценария `scripts/03_marking_animation.jl`, включая построение кадра и обе анимации, получен из литературного исходника.

8.1.1 Анимация классической сети Петри

Обязательный опыт: три философа, горизонт 30. Кадры соответствуют реальным событиям. Дополнительный опыт с арбитром показывает продолжающуюся работу.

```
using DrWatson
@quickactivate "project"
ENV["GKSwstype"] = "100"
using CSV, DataFrames, Plots, Random
include(sourcedir("dining_philosophers.jl"))
using .DiningPhilosophers
```

8.1.1.1 Начальные условия и траектории

```
const N = 3
const TMAX = 30.0
const FPS = 5
const IMAGE_DIR = projectdir("../", "image")
const DATA_DIR = datadir("03_marking_animation")
mkpath(IMAGE_DIR)
mkpath(DATA_DIR)
classical_net, classical_initial = build_classical_network(N)
arbiter_net, arbiter_initial = build_arbiter_network(N)
classical = simulate_stochastic(classical_net, classical_initial,
TMAX;
    rng=MersenneTwister(123))
arbiter = simulate_stochastic(arbiter_net, arbiter_initial, TMAX;
    rng=MersenneTwister(123))
CSV.write(joinpath(DATA_DIR, "classical_trajectory.csv"),
classical.trajectory)
CSV.write(joinpath(DATA_DIR, "arbiter_trajectory.csv"),
arbiter.trajectory)
```

8.1.1.2 Кадр маркировки

Столбец обозначает одну позицию сети, высота — число фишек.

```
function marking_frame(net, row, variant)
    labels = string.(net.place_names)
    values = [row[name] for name in net.place_names]
    bar(labels, values; legend=false, xlabel="Позиции сети",
ylabel="Фишки",
        xrotation=55, ylims=(0, N), color=:steelblue,
        title="$variant, t=$(round(row.time, digits=3))",
        fontfamily="DejaVu Sans", size=(1100, 650),
        bottom_margin=12Plots.mm)
end
```

8.1.1.3 Анимация возникновения deadlock

```
classic_animation = @animate for row in
eachrow(classical.trajectory)
    marking_frame(classical_net, row, "Классическая сеть")
end
gif(classic_animation, joinpath(IMAGE_DIR, "01-animation.gif");
fps=FPS)
```

8.1.1.4 Начальная и последняя маркировки

```
first_frame = marking_frame(classical_net,
first(classical.trajectory), "Начало")
last_frame = marking_frame(classical_net,
last(classical.trajectory), "Остановка")
```

```

static_comparison = plot(first_frame, last_frame; layout=(2, 1),
size=(1100, 950))
savefig(static_comparison, joinpath(IMAGE_DIR, "08-plot.png"))
outcomes = DataFrame(variant=["Классическая", "С арбитром"],
    philosophers=fill(N, 2), horizon=fill(TMAX, 2),
    deadlock=[classical.deadlock, arbiter.deadlock],
    events=[classical.events, arbiter.events],
    last_event=[classical.stop_time, arbiter.stop_time])
CSV.write(joinpath(DATA_DIR, "summary.csv"), outcomes)
show(outcomes; allrows=true, allcols=true)
println()

```

8.1.1.5 Дополнительная анимация с арбитром

```

arbiter_animation = @animate for row in eachrow(arbiter.trajectory)
    marking_frame(arbiter_net, row, "С арбитром")
end
gif(arbiter_animation, joinpath(IMAGE_DIR, "02-animation.gif");
fps=FPS)

```

В классической сети последняя маркировка не допускает ни одного перехода. В модифицированной сети окончание данных обусловлено горизонтом наблюдения, а не взаимной блокировкой.

8.2 Результаты анимационного опыта

Классическая система заблокировалась после 6 событий при $t=2,425484$. В финальной маркировке все три философа находятся в Hungry, все вилки заняты, разрешённых переходов нет. В [рис. 8](#) сопоставлены начальное состояние и последний кадр. Анимация содержит 7 кадров, включая начальный.

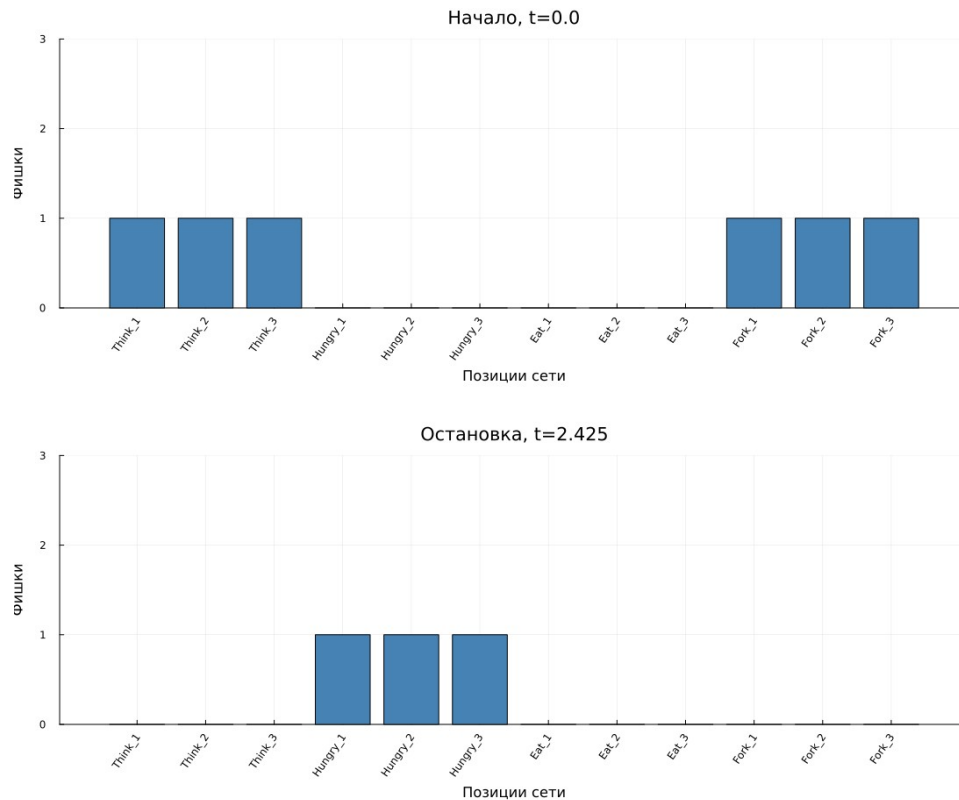


Рисунок 8: Начальная и тупиковая маркировки при $N=3$

Для сравнения тот же сценарий дополнительно создаёт анимацию сети с арбитром — 02-animation.gif. При тех же параметрах она выполняет 36 событий; последнее происходит при $t=27,884388$. Остановка расчёта связана с горизонтом, а не с deadlock. Две траектории и сводная таблица сохранены в data/03_marking_animation.

В [рис. 9](#) показана выполненная ячейка анимационного блокнота. GIF встроен в вывод ячейки; для печатной версии состояния сопоставлены на [рис. 8](#).

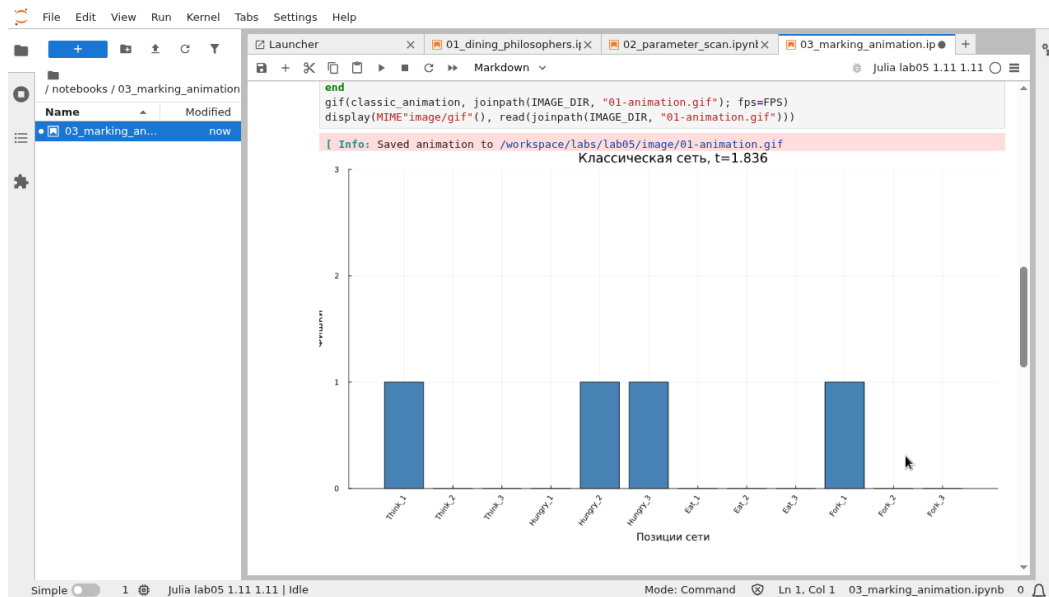


Рисунок 9: Анимационный сценарий в Jupyter

9. Детерминированная аппроксимация

Для непрерывного варианта интенсивность перехода вычислялась по закону действующих масс:

$$a_j(x) = \lambda_j \prod_i x_i^{A_{ij}}, \frac{dx}{dt} = C a(x).$$

ODE-решение не заменяет дискретный анализ достижимости: дробные значения позиций интерпретируются как непрерывные количества. Это приближение среднего поля, а не точное математическое ожидание стохастического процесса: для нелинейных интенсивностей среднее произведения в общем случае не равно произведению средних. Кривые на [рис. 10](#) позволяют сопоставить динамику двух непрерывных моделей.

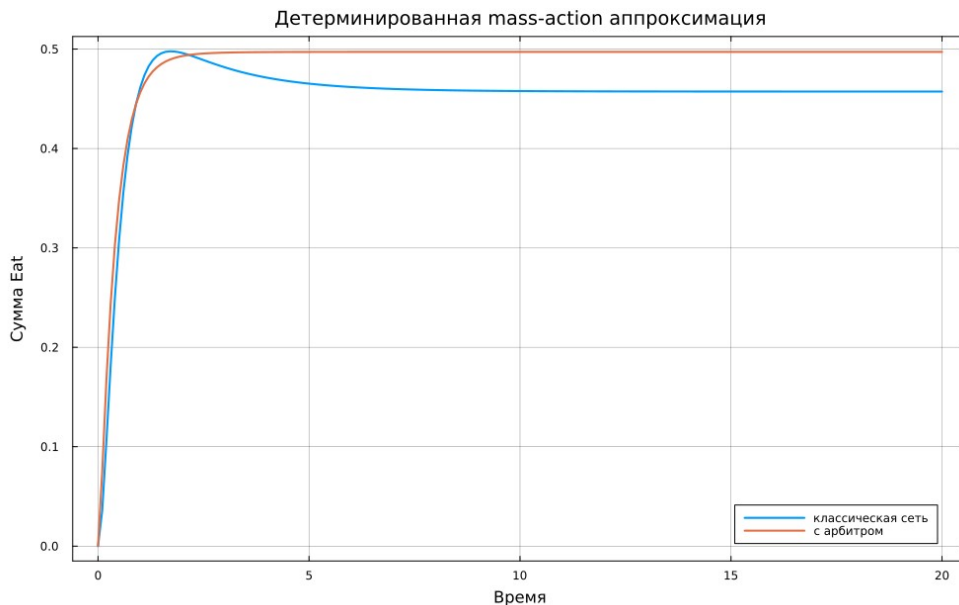


Рисунок 10: Непрерывная аппроксимация суммарной позиции Eat

10. Параметрический эксперимент

Число философов изменялось от 3 до 8, интенсивность GetLeft — от 1,0 до 3,0. Для каждой комбинации выполнено 20 повторов для классической сети и сети с арбитром; всего получено 960 независимых траекторий.

10.1 Документация параметрического сценария

Следующие листинги подключены из QMD-документации scripts/02_parameter_scan.jl: цикл прогонов, сводные показатели и графики.

10.1.1 Обедающие философы: параметрический эксперимент

Исследуется влияние числа философов и интенсивности захвата левой вилки на вероятность deadlock, производительность и равномерность доступа к ресурсам.

```
using DrWatson
@quickactivate "project"
ENV["GKSwstype"] = "100"
using CSV, DataFrames, Plots, Random, Statistics
include(scrdir("dining_philosophers.jl"))
using .DiningPhilosophers
```

10.1.1.1 Параметры серии

```
const TMAX = 50.0
const REPLICATIONS = 20
```

```

const IMAGE_DIR = projectdir("../", "image")
const DATA_DIR = datadir("02_parameter_scan")
mkpath(IMAGE_DIR)
mkpath(DATA_DIR)

```

10.1.1.2 Выполнение 960 независимых траекторий

Для каждого повтора сохраняются исход, время остановки и показатели питания.

```

rows = NamedTuple[]
for N in 3:8, left_rate in (1.0, 1.5, 2.0, 3.0), replication in
1:REPLICATIONS
    rates = default_rates(N; left_rate, right_rate=1.0,
release_rate=1.4)
    for (variant, builder) in (("Классическая",
build_classical_network),
("C арбитром", build_arbiter_network))
        net, initial = builder(N)
        seed = 100_000N + 1_000round(Int, 10left_rate) + replication
+
        (variant == "C арбитром" ? 500 : 0)
        result = simulate_stochastic(net, initial, TMAX;
rates, rng=MersenneTwister(seed))
        push!(rows, (
            philosophers=N,
            left_rate=left_rate,
            replication=replication,
            variant=variant,
            deadlock=result.deadlock,
            stop_time=result.stop_time,
            events=result.events,
            meals=sum(result.meal_counts),
            fairness=jain_fairness(result.meal_counts),
        ))
    end
end

replications = DataFrame(rows)
CSV.write(joinpath(DATA_DIR, "replications.csv"), replications)

```

10.1.1.3 Агрегирование по параметрам

Частота deadlock вычисляется по двадцати повторам одной комбинации.

```

summary = combine(groupby(replications, [:philosophers, :left_rate,
:variant]),
    :deadlock => mean => :deadlock_probability,

```

```

:stop_time => mean => :mean_stop_time,
:events => mean => :mean_events,
:meals => mean => :mean_meals,
:fairness => mean => :mean_fairness)
CSV.write(joinpath(DATA_DIR, "summary.csv"), summary)

println("=== Параметрический эксперимент ===")
println("Число прогонов: $(nrow(replications))")
selected = filter(row -> row.left_rate == 2.0, summary)
show(select(selected, :philosophers, :variant,
:deadlock_probability,
:mean_meals, :mean_fairness); allrows=true, allcols=true)
println()

```

10.1.1.4 Наблюдаемая доля блокировок

```

default(fontfamily="DejaVu Sans", linewidth=2.1, markersize=5,
framestyle=:box, gridalpha=0.22, legendfontsize=8,
guidefontsize=11,
titlefontsize=13, left_margin=7Plots.mm,
bottom_margin=5Plots.mm)

classic = filter(row -> row.variant == "Классическая", summary)
p5 = plot(; xlabel="Число философов", ylabel="Наблюдаемая доля
deadlock",
title="Блокировки классической сети", ylims=(-0.03, 1.03),
size=(1000, 620))
for rate in sort(unique(classic.left_rate))
subset = filter(row -> row.left_rate == rate, classic)
sort!(subset, :philosophers)
plot!(p5, subset.philosophers, subset.deadlock_probability;
marker=:circle, label="λ = $rate")
end
savefig(p5, joinpath(IMAGE_DIR, "06-plot.png"))

```

10.1.1.5 Производительность и равномерность

```

rate_two = filter(row -> row.left_rate == 2.0, summary)
p6a = plot(; xlabel="Число философов", ylabel="Среднее число приёмов
пищи",
title="Производительность")
p6b = plot(; xlabel="Число философов", ylabel="Индекс Джейна",
title="Равномерность доступа", ylims=(0, 1.05))
for variant in ("Классическая", "С арбитром")
subset = filter(row -> row.variant == variant, rate_two)
sort!(subset, :philosophers)
plot!(p6a, subset.philosophers, subset.mean_meals;
marker=:circle, label=variant)
plot!(p6b, subset.philosophers, subset.mean_fairness;

```

```

        marker=:circle, label=variant)
end
p6 = plot(p6a, p6b; layout=(1, 2), size=(1200, 520),
        left_margin=11Plots.mm, bottom_margin=7Plots.mm)
savefig(p6, joinpath(IMAGE_DIR, "07-plot.png"))

```

Серия повторных прогонов отделяет устойчивый эффект структуры сети от случайности отдельных траекторий. Для сети с арбитром наблюдаемая доля глобальных блокировок равна нулю во всей исследованной области параметров.

10.2 Результаты серии параметров

Во всех исследованных комбинациях классическая сеть завершилась deadlock, а сеть с арбитром — ни в одной. Более высокая интенсивность захвата левой вилки ускоряет формирование циклического ожидания: при $N=5$ среднее время блокировки уменьшилось с 6,696 при $\lambda_L=1$ до 1,749 при $\lambda_L=3$. Наблюдаемые частоты представлены на [рис. 11](#); число повторов не даёт основания приравнять их к точным вероятностям для иных условий.

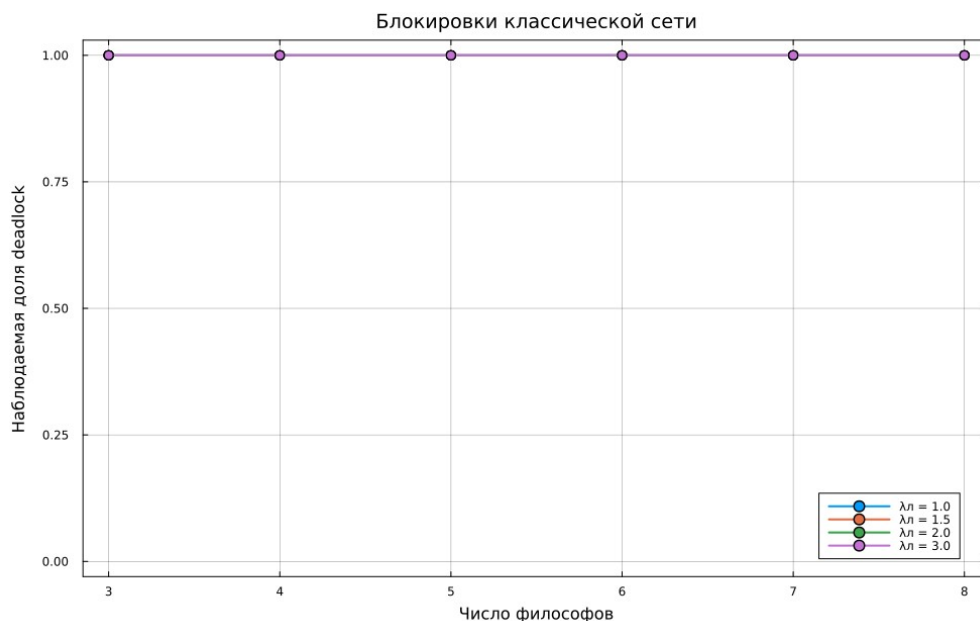


Рисунок 11: Наблюдаемая доля deadlock в классической сети

При $\lambda_L=2$ сеть с арбитром обеспечивает в среднем 29–31 завершённый приём пищи за 50 единиц времени. Индекс Джайна уменьшается с ростом числа философов, но остаётся выше 0,82. Классическая сеть имеет существенно меньшую производительность и равномерность из-за ранней остановки ([рис. 12](#)).

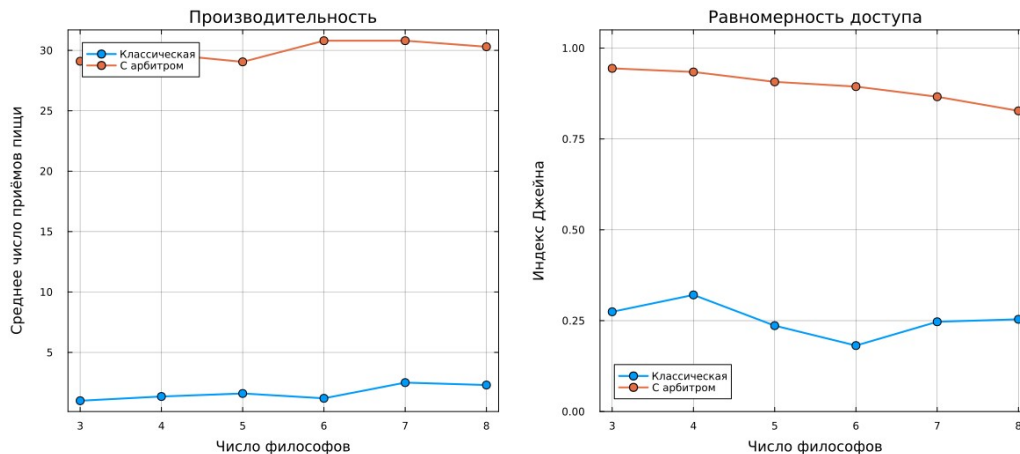


Рисунок 12: Производительность и равномерность доступа

11. Литературное программирование и Jupyter

Исходные сценарии оформлены в стиле Literate.jl. Команда `make tangle` сформировала чистые Julia-файлы, QMD-документацию и четыре блокнота. Блокноты выполнены ядром Julia lab05 1.11, поэтому сохранённые `.ipynb` содержат фактический вывод экспериментов. Базовый сценарий представлен на [рис. 13](#), параметрическая серия — на [рис. 14](#).

Таблица 4: Производные форматы четырёх вычислительных сценариев

Сценарий	Литературный исходник	Производные форматы
Базовая модель	01_dining_philosophers.jl	JL, выполненный IPYNB, QMD, HTML
Серия параметров	02_parameter_scan.jl	JL, выполненный IPYNB, QMD, HTML
Анимация маркировки	03_marking_animation.jl	JL, выполненный IPYNB, QMD, HTML
Анализ CSV	04_comparative_report.jl	JL, выполненный IPYNB, QMD, HTML

Команды выполняются из каталога `project` в следующем порядке. Анализ CSV запускается после базового расчёта, поскольку использует его результаты.

```
make tangle
make run
make notebooks
```

make docs
make test

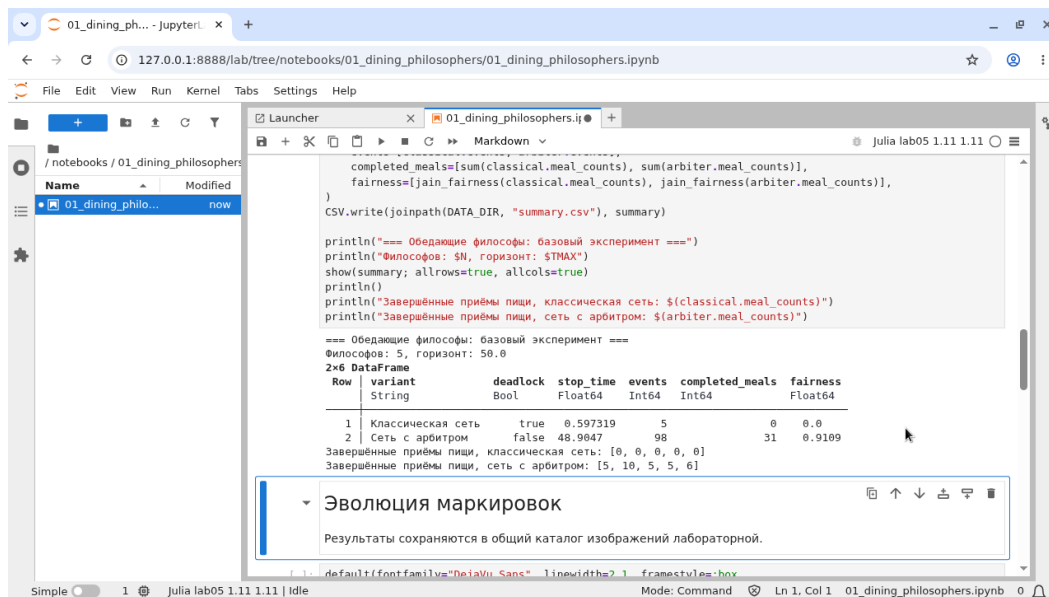


Рисунок 13: Выполненный базовый блокнот Jupyter



Рисунок 14: Выполненный параметрический блокнот Jupyter

12. Проверка реализации

Тесты охватывают размерности матриц, начальную маркировку, список разрешённых переходов, сохранение неотрицательности, известную тупиковую маркировку, различие классической сети и сети с арбитром, ODE-траекторию и индекс Джайна.

```

@testset "Known deadlock marking" begin
    net, marking = build_classical_network(5)
    marking .= 0
    marking[6:10] .= 1
    @test detect_deadlock(net, marking)
end

@testset "Stochastic models" begin
    rates = default_rates(5)
    classical_net, classical_initial = build_classical_network(5)
    arbiter_net, arbiter_initial = build_arbiter_network(5)
    classical = simulate_stochastic(classical_net,
        classical_initial, 50.0; rates, rng=MersenneTwister(202605))
    arbiter = simulate_stochastic(arbiter_net,
        arbiter_initial, 50.0; rates, rng=MersenneTwister(202605))
    @test classical.deadlock
    @test !arbiter.deadlock
    @test sum(arbiter.meal_counts) > 0
end

```

Листинг 4 — Проверка deadlock и арбитра в test/runtests.jl

Дополнительно проверяется сохранение состояния каждого философа: $Think_i + Hungry_i + Eat_i = 1$. Для каждой вилки проверяется ресурсный инвариант $Fork_i + Hungry_i + Eat_i + Eat_{i-1} = 1$, где индексы берутся циклически. Проверки выполнены по всем строкам траекторий при $N=3$ и $N=5$ для обеих сетей. Они обнаруживают потерю ресурса или одновременную выдачу одной вилки двум несовместимым состояниям. Отдельные тесты сверяют анализ CSV, классическую анимационную траекторию и наличие трёх производных форматов у всех сценариев.

Все 56 проверок завершились успешно. Дополнительно проверены целостность итоговых форматов и согласованность данных, включённых в таблицы отчёта. Итог запуска тестов показан на [рис. 15](#).

```
dakuokkonen@lab05:/workspace/labs/lab05/project$ make test
julia --project=. test/runtests.jl
Test Summary: | Pass Total Time
Petri net structure | 5 5 0.7s
Test Summary: | Pass Total Time
Conservation on complete trajectories | 12 12 1.4s
Test Summary: | Pass Total Time
Classical animation experiment | 3 3 0.1s
Test Summary: | Pass Total Time
Saved CSV analysis | 6 6 2.0s
Test Summary: | Pass Total Time
Literate outputs for every scenario | 12 12 0.0s
Test Summary: | Pass Total Time
Transition firing and invariants | 6 6 0.0s
Test Summary: | Pass Total Time
Known deadlock marking | 1 1 0.0s
Test Summary: | Pass Total Time
Stochastic models | 6 6 0.6s
Test Summary: | Pass Total Time
ODE and fairness | 5 5 2.1s
dakuokkonen@lab05:/workspace/labs/lab05/project$
```

Рисунок 15: Успешное выполнение автоматических тестов

13. Выводы

1. Классическая сеть Петри воспроизводит взаимную блокировку: при базовом запуске пять философов одновременно захватили левые вилки, после чего множество разрешённых переходов стало пустым.
2. Позиция-арбитр с $N - 1$ фишками устраняет глобальный deadlock, поскольку не допускает одновременного включения всех процессов в цикл ожидания.
3. В базовой траектории сеть с арбитром выполнила 31 приём пищи и достигла индекса равномерности 0,9109.
4. В серии из 960 прогонов наблюдаемая доля блокировок составила 1 для классической сети и 0 для сети с арбитром. Эти частоты относятся к выбранным параметрам и горизонту моделирования, а не ко всем возможным траекториям.
5. Стохастическая модель отражает отдельные траектории событий, а детерминированная mass-action система описывает усреднённую динамику; эти представления дополняют друг друга, но не взаимозаменяемы.
6. Проект полностью представлен Julia-исходниками, данными, выполненными Jupyter-блокнотами, документацией Quarto и автоматическими тестами.

ИСТОЧНИКИ

1. Petri C. A. Kommunikation mit Automaten: phdthesis. Universität Hamburg, 1962.
2. Peterson J. L. [Petri Net Theory and the Modeling of Systems](#) // ACM Computing Surveys. 1977. Т. 9, № 3. С. 223–252.
3. Gillespie D. T. [Exact Stochastic Simulation of Coupled Chemical Reactions](#) // The Journal of Physical Chemistry. 1977. Т. 81, № 25. С. 2340–2361.